

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Curso: Análisis Mecánico
DocIng. Oscar Maldonado
Grupo: 12

Fase 1: Investigación

Integrantes:

Amilcar Josué Escobedo Morales 202030274

Marlon Ivan Carreto Rivera 201230088

Kenny Werner Gamaliel Mux Saníc 202132727

Guía de Desafío: "La Danza de los Péndulos con IA"

Física del Péndulo:

1.1 ¿Cuál es la ecuación de movimiento para la posición angular $\theta(t)$ de un péndulo simple para ángulos pequeños?

1. Análisis de la Ecuación de Movimiento: Solución para Ángulos Pequeños

Basado en el análisis profundo y la integración de los recursos proporcionados (OpenStax, EHU, UCM, Khan Academy y los videos de física universitaria), se presenta la respuesta técnica y fundamentada para la investigación. Esta explicación desglosa la ecuación, su origen físico y las condiciones matemáticas que la hacen válida.

Para un péndulo simple idealizado, cuando las oscilaciones son pequeñas (régimen lineal), la posición angular $\theta(t)$ está dada por:

$$\theta(t) = \theta_{max} \cos \left(\sqrt{\frac{g}{L}} \cdot t \right) \quad (1)$$

1.1. 1. Desglose de las Variables (Análisis Dimensional)

Según los textos de OpenStax y la Guía UCM, cada término tiene un significado físico estricto:

$\theta(t)$ (**Posición Angular**): Es el ángulo del hilo respecto a la vertical en un instante t . Su valor oscila entre $+\theta_{max}$ y $-\theta_{max}$.

θ_{max} (**Amplitud**): Es el desplazamiento angular máximo inicial. Representa la condición inicial del sistema suponiendo que se suelta el péndulo desde el reposo en $t = 0$.

$\cos$ (**Función Armónica**): La función coseno dicta la naturaleza periódica y repetitiva del movimiento.

$\sqrt{g/L}$ (**Frecuencia Angular Natural, ω**): Este término es el "motor" del oscilador.

- g (**Gravedad**): Actúa como la fuerza restauradora. A mayor gravedad, más rápida es la oscilación (numerador).
- L (**Longitud**): Representa la inercia rotacional. A mayor longitud, más lenta es la oscilación (denominador).

t (**Tiempo**): La variable independiente que hace evolucionar el sistema.

1.2. 2. Fundamentación Física y Matemática (Origen de la Ecuación)

Para que el modelo sea funcional, es crucial entender el origen de la fórmula, basándonos en la deducción matemática de la EHU y los diagramas de fuerzas de Khan Academy.

A. La Ecuación Diferencial (El "Motor" Matemático)

El movimiento no empieza con el coseno, sino con la Segunda Ley de Newton para la rotación ($\tau = I\alpha$). La fuerza que mueve al péndulo es la componente del peso tangencial a la trayectoria:

$$F_t = -mg \sin(\theta) \quad (2)$$

Esto lleva a la ecuación diferencial no lineal:

$$\frac{d^2\theta}{dt^2} = -\frac{g}{L} \sin(\theta) \quad (3)$$

B. La Aproximación de Ángulo Pequeño (La Clave del Modelo)

Aquí se aplica la restricción de "ángulos pequeños". Como explican los recursos técnicos (César Izquierdo, OpenStax), si $\theta < 15^\circ$ (aprox. 0,26 rad), entonces matemáticamente:

$$\sin(\theta) \approx \theta \quad (4)$$

Esto transforma la ecuación compleja en una Ecuación de Movimiento Armónico Simple (MAS):

$$\frac{d^2\theta}{dt^2} + \frac{g}{L} \theta = 0 \quad (5)$$

C. La Solución Integral

La solución matemática a esta ecuación diferencial lineal es una combinación de senos o cosenos. Dado que el péndulo suele soltarse desde un extremo ($v_0 = 0$), la solución correcta es el Coseno:

$$\theta(t) = \theta_{max} \cos(\omega t) \quad (6)$$

Sustituyendo $\omega = \sqrt{g/L}$, obtenemos la fórmula final (1).

1.3. 3. Validación y Comportamiento del Modelo

Según los datos de validación del laboratorio de la UCM, esta ecuación implica tres realidades físicas que el modelo debe respetar:

- **Isocronismo:** El periodo T es independiente de la amplitud θ_{max} (en régimen de ángulos pequeños). El péndulo tarda lo mismo en oscilar 2° que 5° .
- **Independencia de la Masa:** La variable m no aparece en la ecuación final. Un péndulo de plomo oscila igual que uno de madera si L es constante.

- **Relación con el Periodo:** La ecuación confirma que el periodo es:

$$T = \frac{2\pi}{\omega} = 2\pi\sqrt{\frac{L}{g}} \quad (7)$$

Física del Péndulo:

1.2 El secreto de la Ola

Para que los péndulos bailen en forma de "serpiente" luego se vuelvan a juntar, sus longitudes no son aleatorias.

2. Introducción y Fundamento Físico

Basado en los principios de dinámica del péndulo simple establecidos en la literatura técnica (OpenStax, EHU), sabemos que el periodo de oscilación T para amplitudes pequeñas es independiente de la masa y depende exclusivamente de la longitud L y la gravedad g .

La ecuación fundamental del periodo es:

$$T = 2\pi\sqrt{\frac{L}{g}} \quad (8)$$

Despejando para la longitud L , obtenemos la relación base para el diseño:

$$L = \frac{gT^2}{4\pi^2} \quad (9)$$

3. Principio de la Onda de Péndulo (Aliasing Temporal)

Para generar el efecto visual de onda viajera.º "serpiente" que converge y diverge, no se deben variar las longitudes de manera lineal. El principio físico requiere una **cuantización lineal de las frecuencias**.

Se define un tiempo de ciclo total t_{ciclo} (tiempo de sincronización) en el cual todos los péndulos del sistema completan un número entero de oscilaciones y regresan a la fase inicial simultáneamente.

Sean las variables del sistema:

- t_{ciclo} : Tiempo total del ciclo de la coreografía (ej. 60 segundos).
- N : Número base de oscilaciones del péndulo más largo durante t_{ciclo} .
- n : Índice del péndulo ($n = 0, 1, 2, \dots, k$).
- T_n : Periodo del péndulo n .

La condición de diseño dicta que el péndulo n debe realizar $N + n$ oscilaciones exactas en el tiempo t_{ciclo} . Por lo tanto, el periodo requerido para el péndulo n es:

$$T_n = \frac{t_{ciclo}}{N + n} \quad (10)$$

4. Derivación de la Ecuación de Longitud

Sustituyendo la Ecuación (10) en la Ecuación (9), obtenemos la fórmula maestra para calcular la longitud de cada péndulo en la serie:

$$L_n = \frac{g}{4\pi^2} \left(\frac{t_{ciclo}}{N + n} \right)^2 \quad (11)$$

Reordenando los términos para facilitar el cálculo computacional:

$$L_n = g \left(\frac{t_{ciclo}}{2\pi(N + n)} \right)^2 \quad (12)$$

Donde:

- L_n : Longitud del hilo para el péndulo n [metros].
- g : Aceleración de la gravedad ($\approx 9,81 \text{ m/s}^2$).
- $N + n$: Frecuencia total de oscilaciones en el ciclo para el péndulo actual.

5. Ejemplo Numérico para Validación del Modelo

Para implementar un modelo funcional (simulación o prototipo) con los siguientes parámetros estándar:

- Ciclo de sincronización: $t_{ciclo} = 60 \text{ s}$
- Oscilaciones base (péndulo más largo): $N = 51$
- Gravedad: $g = 9,81 \text{ m/s}^2$

Cálculo para el Péndulo 0 (Más largo)

$$n = 0 \rightarrow \text{Oscilaciones} = 51$$

$$L_0 = 9,81 \left(\frac{60}{2\pi(51)} \right)^2 \approx 0,3474 \text{ m}$$

Cálculo para el Péndulo 1

$$n = 1 \rightarrow \text{Oscilaciones} = 52$$

$$L_1 = 9,81 \left(\frac{60}{2\pi(52)} \right)^2 \approx 0,3342 \, m$$

6. Herramientas de Python

Librerías Principales

A continuación se presenta un resumen de las librerías esenciales para cálculos matemáticos y vectores.

Librería	Enfoque Principal	Función Clave
NumPy	Arrays Numéricos y Álgebra Lineal	Proporciona la estructura de datos ndarray (matrices eficientes) y operaciones vectorizadas de alta velocidad. Es la base de todo.
SciPy	Algoritmos Científicos Avanzados	Ofrece módulos de alto nivel para problemas estándar: integración, optimización, estadística, transformada de Fourier.
Matplotlib	Gráficos y Visualización	Crea todo tipo de gráficos estáticos y dinámicos para visualizar datos y funciones matemáticas.
Pandas	Análisis y Estructuras de Datos	Permite cargar, limpiar y manipular datos en estructuras tipo tabla (DataFrame), esencial para pre-procesar datos para el cálculo.
SymPy	Matemáticas Algebraicas	Realiza cálculo simbólico: derivadas, integrales y resolución de ecuaciones, devolviendo la expresión matemática, no un número.
StatsModels	Modelos Estadísticos y Econometría	Análisis estadístico profundo más allá de SciPy (e.j., modelos de regresión avanzados).
Scikit-learn	Aprendizaje Automático (ML)	Conjunto de algoritmos de ML que se basan en optimización y álgebra lineal para clasificación, regresión y clustering.
Dask	Procesamiento Paralelo y Distribuido	Permite escalar las operaciones de NumPy y Pandas a conjuntos de datos demasiado grandes para la memoria de una sola máquina.

Librerías para Graficar

Consejos del “Profe” sobre cuándo utilizar cada librería.

Librería	Tipo de Gráfico	¿Cuándo usarla?
Matplotlib	Estático (Principalmente)	Cuando necesitas control total sobre cada píxel. Es el estándar para publicaciones científicas y la base de otras librerías.
Seaborn	Estático (Estadístico)	Cuando quieres gráficos bonitos y estadísticos (mapas de calor, distribuciones) con muy pocas líneas de código. Se basa en Matplotlib.
Plotly	Interactivo (Web)	Cuando necesitas gráficos con zoom, tooltips (información al pasar el mouse) o para dashboards web. ¡Son espectaculares!
Bokeh	Interactivo (Alto Rendimiento)	Similar a Plotly, pero brilla cuando tienes grandes volúmenes de datos que necesitas visualizar interactivamente en un navegador.
Altair	Declarativo	Si prefieres definir “qué” quieres graficar en lugar de “cómo”. Tiene una sintaxis muy lógica y limpia.
Pandas (.plot)	Estático (Rápido)	Para un vistazo rápido. No es una librería aparte, sino el comando .plot() integrado en Pandas (usa Matplotlib por debajo).

Creación de Animaciones

Tabla comparativa de herramientas para animación en Python.

Librería	¿Preinstalada?	Tipo de Animación	Dificultad	Mejor uso
Matplotlib	SÍ	Video (MP4/JS)	Media	Física, Cálculo, Ecuaciones Diferenciales.
Plotly	SÍ	Web Interactiva	Muy Fácil	Evolución de datos estadísticos, gráficas de dispersión animadas.
IPyWidgets	SÍ	Control Manual	Fácil	Entender cómo cambian parámetros (enseñanza).
Manim	NO*	Calidad Cine	Alta	Videos matemáticos profesionales (estilo 3Blue1Brown).

7. Animaciones en Google Colab con Matplotlib

El proceso se divide en tres pasos principales: la preparación de la figura, la definición de la función de animación, y la visualización.

7.1. 1. Preparación de la Figura

Primero, necesitas una figura de Matplotlib donde se dibujará la animación.

```
1 import matplotlib.pyplot as plt
2 from matplotlib.animation import FuncAnimation
3 from IPython.display import HTML
4 import numpy as np
5
6 # 1. Configurar la figura y los ejes
7 fig, ax = plt.subplots()
8
9 # Crear un objeto lineal inicial que se actualizará
10 line, = ax.plot([], [], lw=2)
11
12 # Establecer los límites iniciales (necesario para el gráfico)
13 ax.set_xlim(0, 2 * np.pi)
14 ax.set_ylim(-1.1, 1.1)
15
```

Listing 1: Configuración inicial

7.2. 2. ¿Qué es FuncAnimation?

FuncAnimation es una clase dentro del módulo `matplotlib.animation` que se utiliza para crear animaciones llamando repetidamente a una función definida por el usuario. Es el motor de tu animación.

Parámetros Clave:

- **fig:** La figura de Matplotlib donde se dibujará la animación.
- **func:** Una función que se llama en cada frame. Recibe el índice del frame (*i*) como argumento.
- **frames:** El número total de frames a generar.
- **interval:** Tiempo de retraso entre frames en ms.
- **blit:** Si es True, optimiza el dibujo (solo redibuja lo que ha cambiado).

```
1 def update(frame):
2     # La función 'update' se llama 'frames' veces
3     # 'frame' toma valores de 0 hasta 'frames' - 1.
4
5     # Generar datos para el frame actual (una onda seno que se desplaza)
6     x = np.linspace(0, 2 * np.pi, 100)
7     y = np.sin(x + frame / 10.0) # El desplazamiento depende del frame
8
9     # Actualizar los datos del objeto lineal (line)
10    line.set_data(x, y)
11
12    # Debe devolver una tupla de objetos Matplotlib modificados (para blit
    =True)
```

```

13     return (line,)
14
15 # 2. Crear el objeto FuncAnimation
16 # 100 frames, con 20 ms de intervalo (50 frames/s)
17 anim = FuncAnimation(fig, update, frames=100, interval=20, blit=True)
18

```

Listing 2: Función de actualización

7.3. 3. Visualización (HTML display)

En entornos como Google Colab, se utiliza `anim.to_html5_video()` para generar una etiqueta `<video>` de HTML5.

```

1 # 3. Mostrar la animación como un video HTML5
2 # Cierra la figura de Matplotlib para evitar que se muestre como imagen
  estática
3 plt.close(fig)
4 HTML(anim.to_html5_video())
5

```

Listing 3: Mostrar animación

8. Resumen del Código Completo

```

1 import matplotlib.pyplot as plt
2 from matplotlib.animation import FuncAnimation
3 from IPython.display import HTML
4 import numpy as np
5
6 # 1. Inicialización
7 fig, ax = plt.subplots()
8 x = np.linspace(0, 2 * np.pi, 100)
9 line, = ax.plot(x, np.sin(x))
10 ax.set_xlim(0, 2 * np.pi)
11 ax.set_ylim(-1.1, 1.1)
12
13 # 2. Función de actualización por frame
14 def update(frame):
15     y = np.sin(x + frame / 10.0)
16     line.set_ydata(y)
17     return (line,)
18
19 # 3. Creación de la animación (100 frames, 20ms de intervalo)
20 anim = FuncAnimation(fig, update, frames=100, interval=20, blit=True)
21
22 # 4. Visualización en Colab
23 plt.close(fig) # Importante para evitar la figura estática
24 HTML(anim.to_html5_video())
25

```

Listing 4: Código completo para Google Colab

9. Descripción Detallada del Proceso

9.1. El Motor: `matplotlib.animation.FuncAnimation`

`FuncAnimation` gestiona el ciclo de vida de la animación mediante:

- **Inicialización (`init_func`):** Dibuja los elementos estáticos.
- **Actualización (`func`):** Se llama repetidamente. Recibe el frame actual (i) y actualiza los datos (x e y) usando métodos como `line.set_ydata()`.
- **Blit=True:** Técnica que solo redibuja las regiones que han cambiado, aumentando drásticamente la velocidad.

9.2. Conversión a Video y Despliegue

Dado que Google Colab no tiene un backend de visualización en tiempo real eficiente para esto:

1. Se generan todos los frames en memoria.
2. `anim.to_html5_video()` convierte los frames a imágenes y los codifica en video (H.264/VP9).
3. Se genera una cadena HTML base64.
4. `IPython.display.HTML` inyecta esta cadena en el notebook para que el navegador la reproduzca.